OPERATING SYSTEMS

CS X61

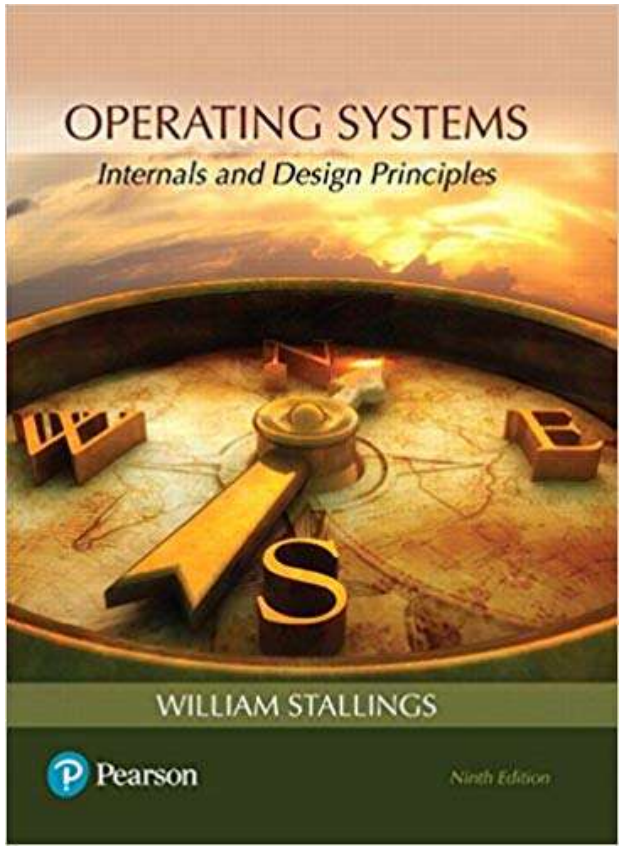
Aims of the Course

- Study the principles underlying the design and the implementation of contemporary computer operating systems.
- Understanding the major components of an OS
- Introduction to the design of operating systems, concurrent processing, synchronization, and storage management policies.
- Current trends in Operating Systems

Course Outline

- Introduction
- Operating Systems Structure
- Processes and threads
- Concurrency management
- Memory Management techniques
- Process Scheduling
- Disk Scheduling
- Device Management
- File Systems
- Protection & Security
- Virtual machines
- Case studies: UNIX, Linux, Windows, Android

Course Textbook



Title: Operating Systems: Internals and Design Principles (9th Edition), 2018

Authors: William Stallings

Publisher: Pearson

ISBN: 978-0134670959

References:

- Modern Operating Systems, Andrew Tanenbaum, Pearson 4th Edition 2014, ISBN: 978-1292061429
- Operating System Concepts, Silberschatz, Galvin, and Gagne, John Wiley, 9th Edition 2012, ISBN: 978-1118063330

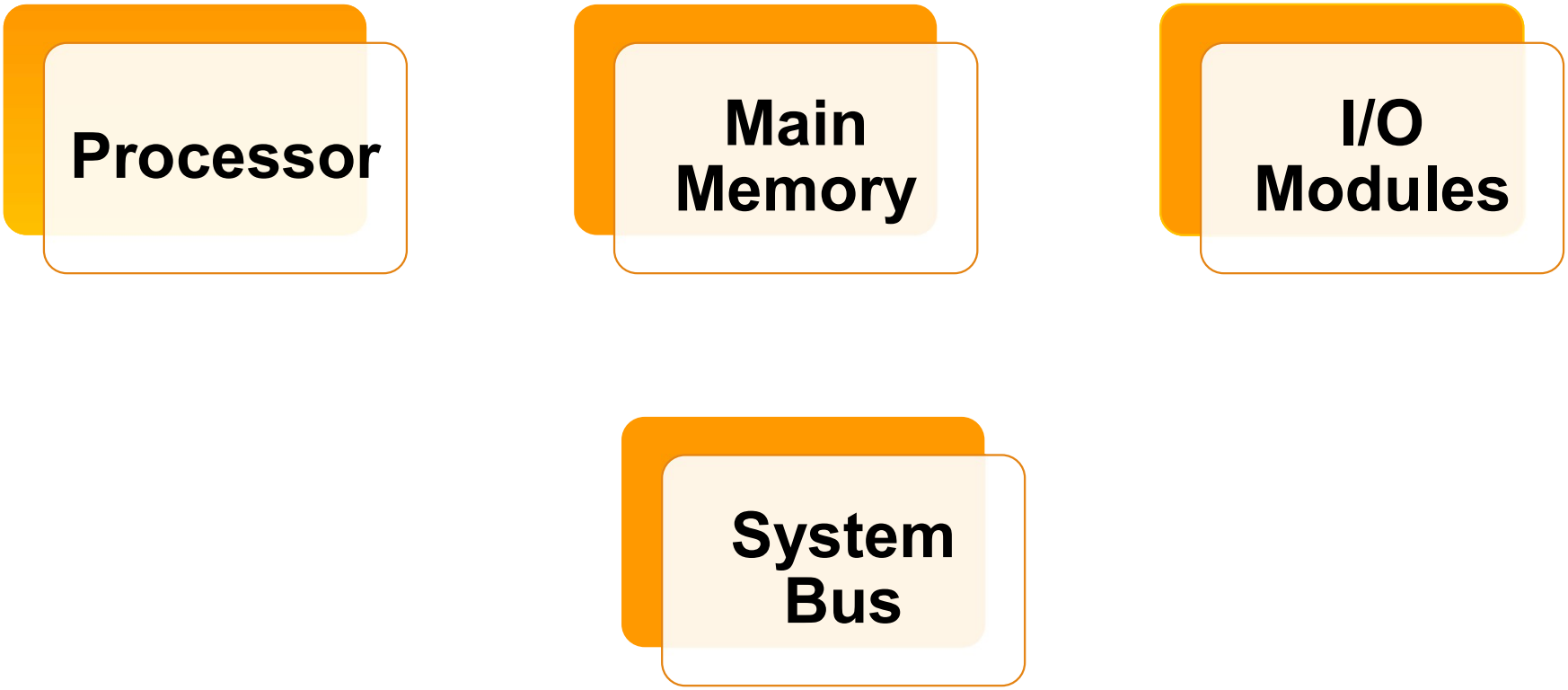
- Grading
  - Total = 100
  - Final Exam = 50
  - Year Work = 50

  (Midterm + Sheets + Lab Assignments + Project)
- Lectures
  - Monday 2nd lecture: 10:20-12:00
  - Attendance: 75%
- Section
  - Teaching Assistant: Eng. Sajed El Morsy

Operating System

- Exploits the hardware resources of one or more processors to provides a set of services to system users
- Manages secondary memory and I/O devices
- Provides support for networking and communications

Basic Elements



Provides for communication among processors, main memory, and I/O modules

Processor

Controls the operation of the computer

Performs the data processing functions

Referred to as the *Central Processing Unit* (CPU)

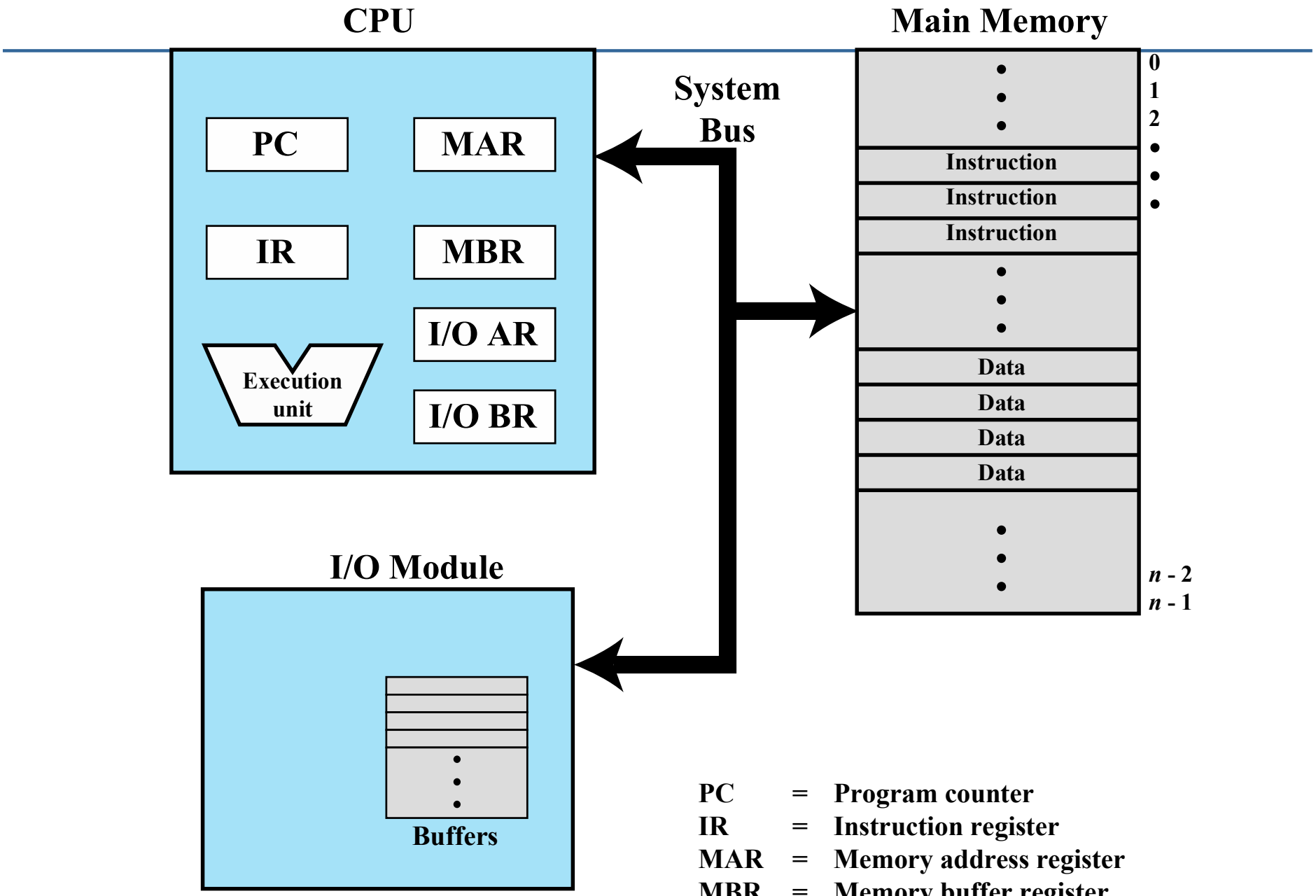
Main Memory

- Stores data and programs
- Typically volatile
  - Contents of the memory is lost when the computer is shut down
- Referred to as *real memory* or *primary memory*

I/O Modules

Move data between the computer and its external environment

- Secondary memory devices (e.g. disks)
- Communications equipment
- Terminals



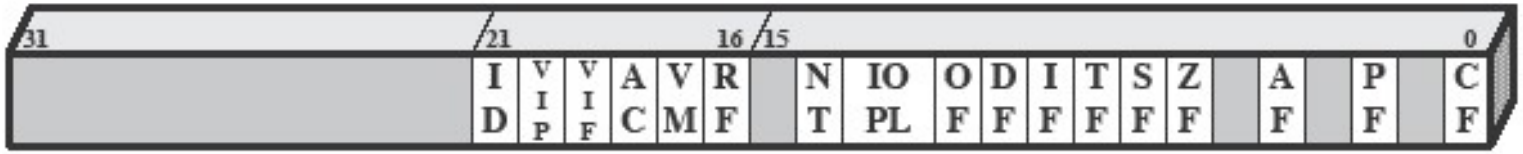
PC = Program counter
IR = Instruction register
MAR = Memory address register
MBR = Memory buffer register
I/O AR = Input/output address register
I/O BR = Input/output buffer register

Processor Registers

- Faster and smaller than main memory
- User-visible registers
  - Minimize main memory access by optimizing register use
  - May be referenced by machine language
  - Available to all programs - application and system programs
    - Data Registers – can be changed by user
    - Address Registers – memory addresses of data and instructions
      - Index registers
      - Segment pointers – used in memory addressing
      - Stack Registers – user / supervisor stacks
- Control and status registers May or may not be visible
    - Program Counter (PC) – address of next instruction
    - Instruction Register (IR) – most recently fetched instruction
    - Condition Codes – results of operations
    - MAR/MBR – memory reference registers
    - Program Status Word (PSW) – condition codes, interrupts, mode

Program Status Word (PSW)

- All processor designs include a PSW register
- PSW contain condition codes plus other status information
- A good example of a PSW is that on Pentium processors



ID	=	Identification flag	DF	=	Direction flag
VIP	=	Virtual interrupt pending	IF	=	Interrupt enable flag
VIF	=	Virtual interrupt flag	TF	=	Trap flag
AC	=	Alignment check	SF	=	Sign flag
VM	=	Virtual 8086 mode	ZF	=	Zero flag
RF	=	Resume flag	AF	=	Auxiliary carry flag
NT	=	Nested task flag	PF	=	Parity flag
IOPL	=	I/O privilege level	CF	=	Carry flag
OF	=	Overflow flag			

Figure 3.12 Pentium II EFLAGS Register

Microprocessor

- Invention that brought about desktop and handheld computing
- Contains a processor on a single chip
- Fastest general-purpose processors
- Multiprocessors
  - Each chip (socket) contains multiple processors (cores)

Instruction Execution

- A program consists of a set of instructions stored in memory

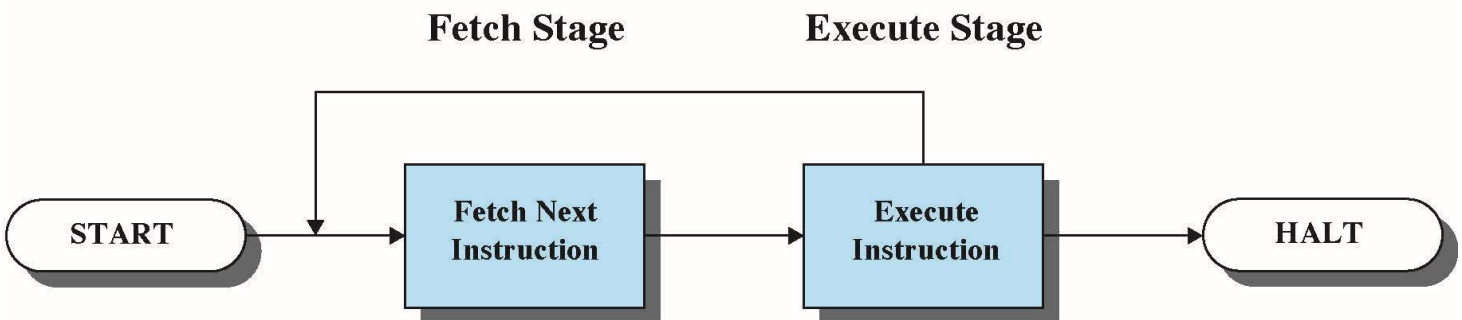
Processor reads (**fetches**) instructions from memory

Processor **executes** each instruction

Two steps

Instruction Execution

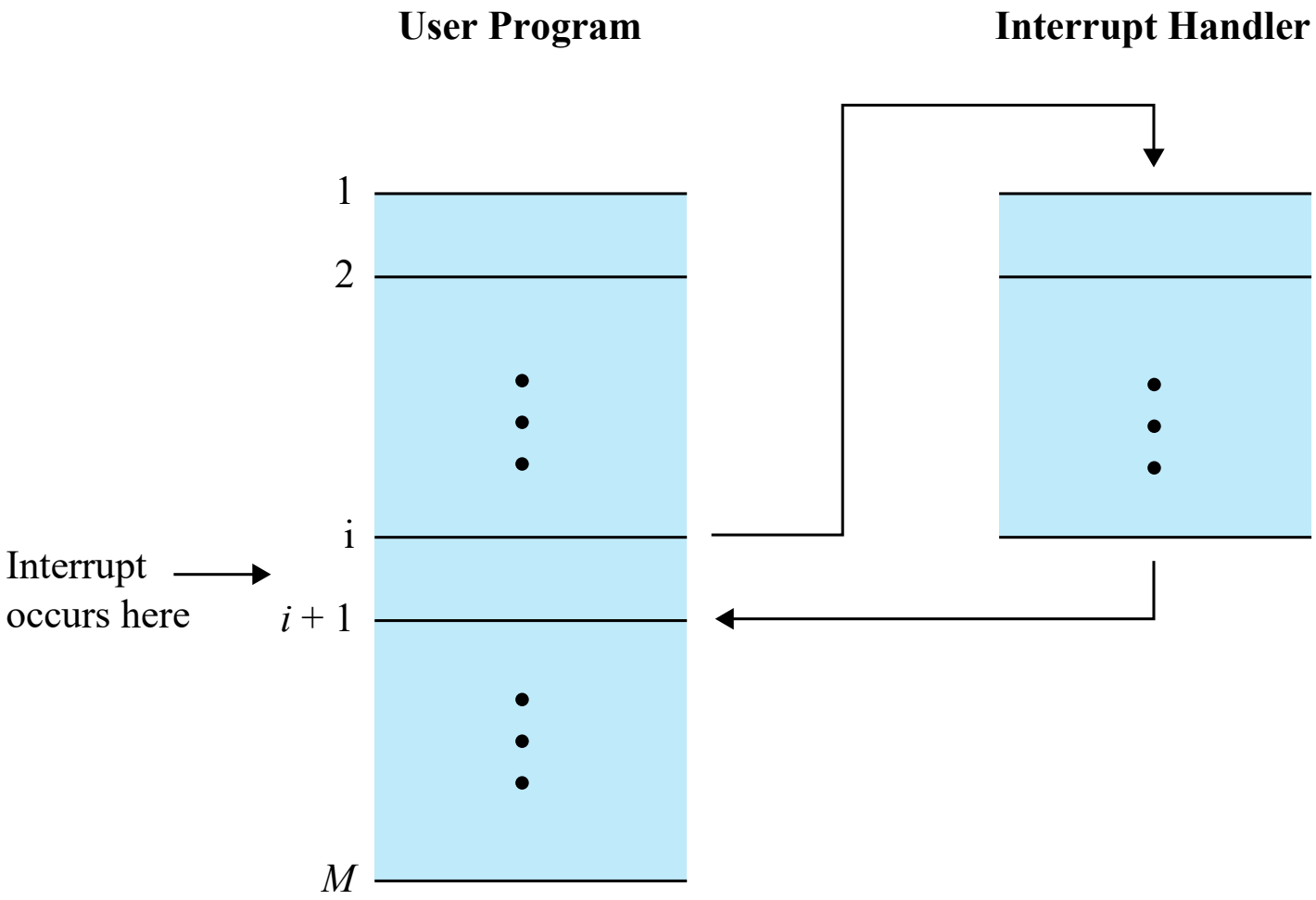
- A program consists of a set of instructions stored in memory
- The processor fetches the instruction from memory where the Program Counter (PC) is pointing
- PC is incremented after each fetch
- Fetched instruction is placed in the Instruction Register (IR) and then decoded for execution
- Types of instructions
  - *Processor-memory* - transfer data between processor and memory
  - *Processor-I/O* - data transferred to or from a peripheral device
  - *Data processing* - arithmetic or logic operation on data
  - *Control* - alter sequence of execution (e.g. jump, branch etc.)



Interrupts

- Mechanism by which other modules may interrupt the normal sequencing of the processor
- Provided to improve processor utilization
  - Most I/O devices are slower than the processor
  - Example
    - a PC that operates at 1 GHz, allow ≈ 10^9 instructions per second
    - A typical hard disk has a rotational speed of 7200 revolutions per minute for a half-track rotation time of 4 ms, which is 4 million times slower than the processor.
  - Processor must pause to wait for device
  - Wasteful use of the processor

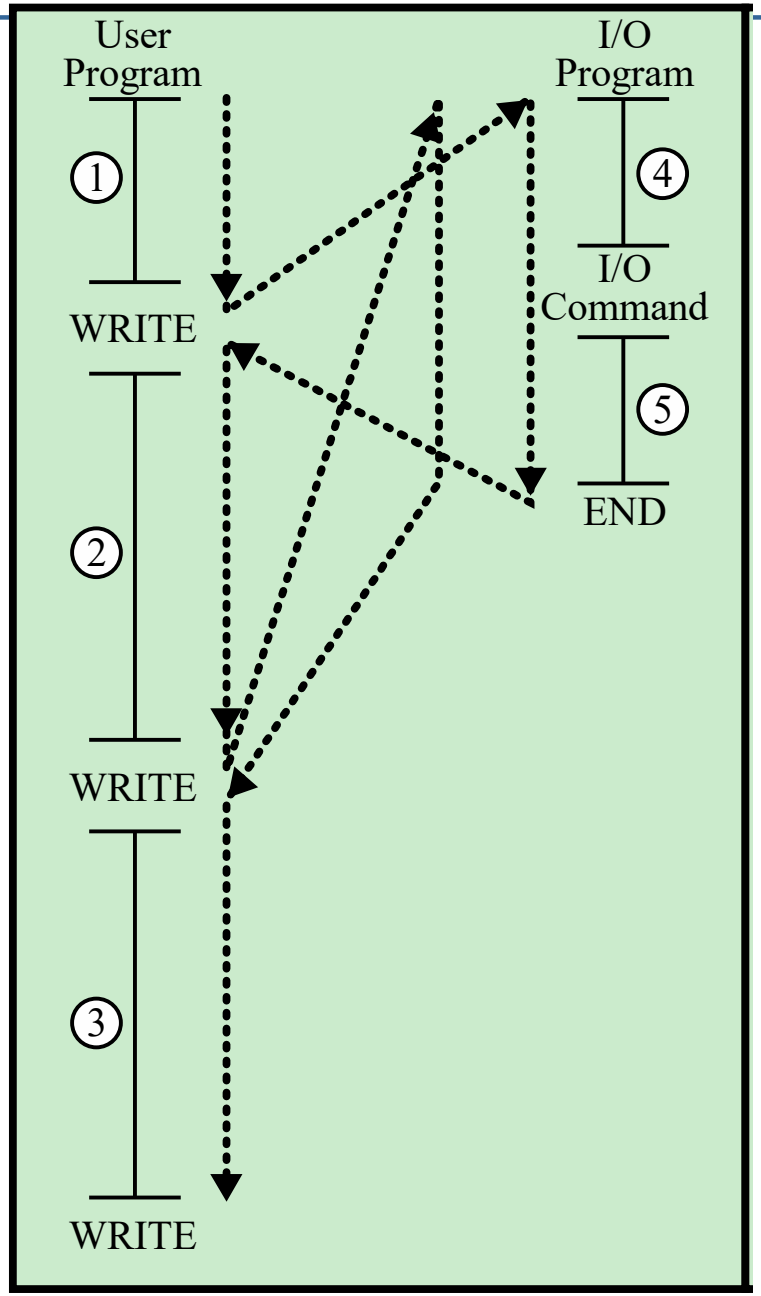
Interrupt Handler



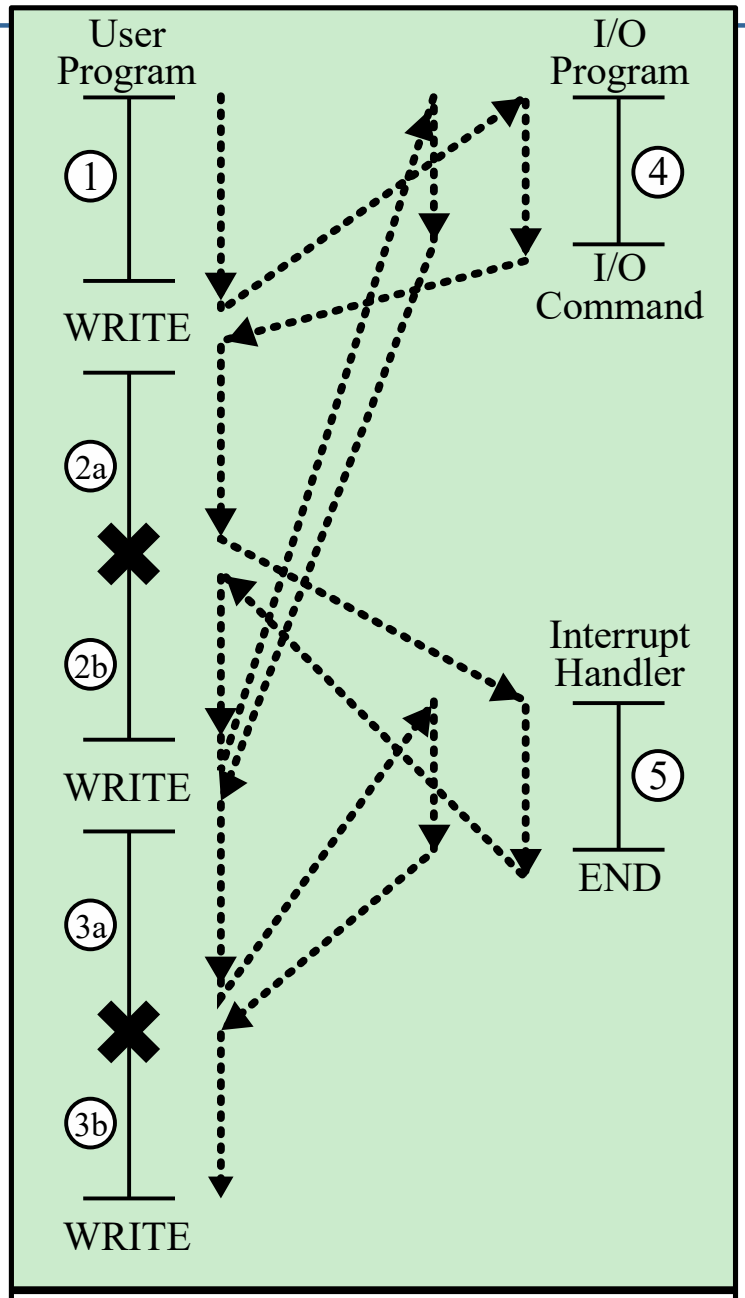
Interrupt Handler (ISR – Interrupt Service Routine)

Whenever there is an interrupt, control is transferred to this program. It determines the nature of the interrupt and performs the necessary actions to handle it.

Flow of Control

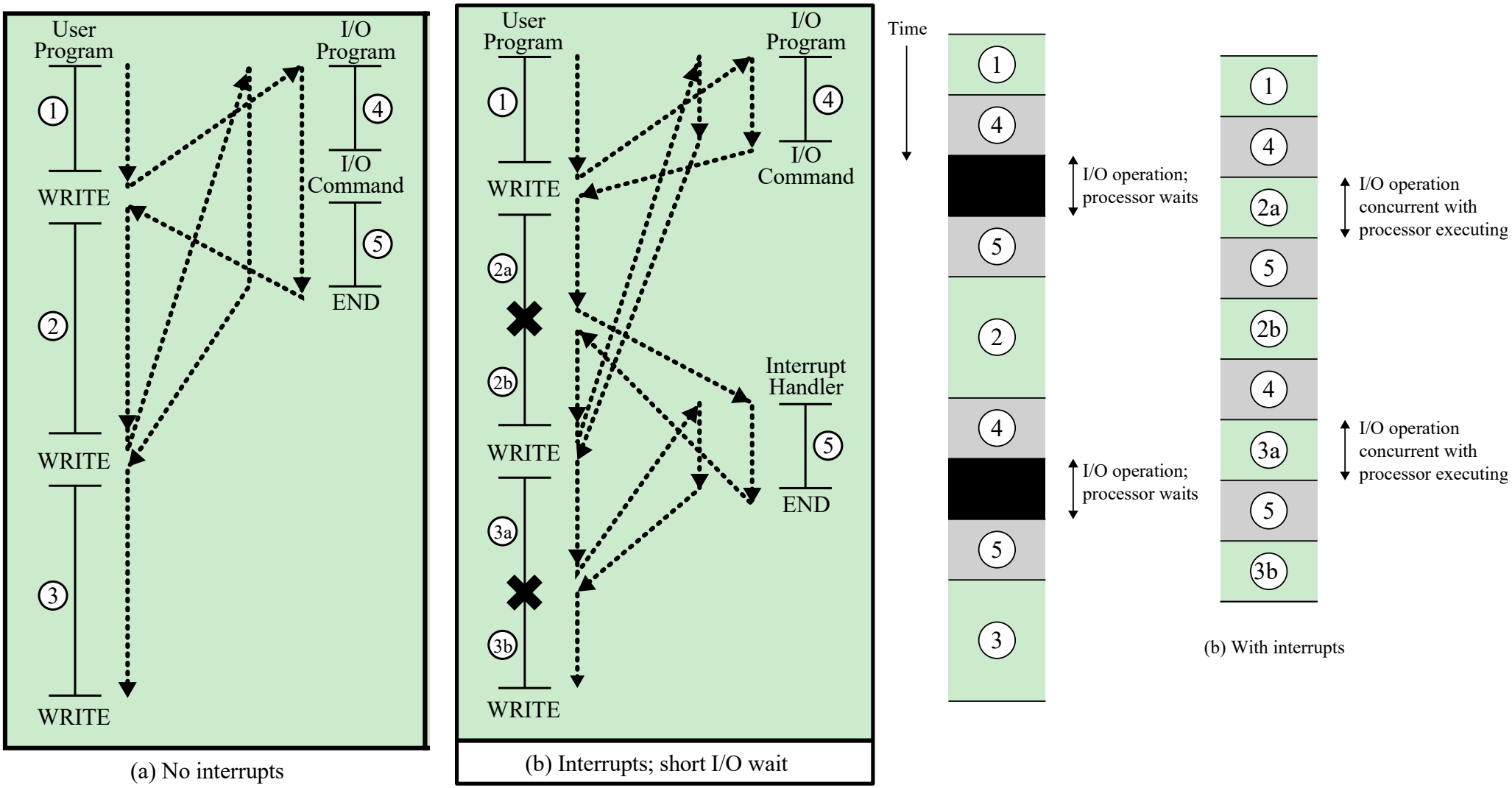


(a) No interrupts



(b) Interrupts; short I/O wait

Short I/O Wait



Interrupt Cycle

- At the end of each cycle, processor checks for interrupts
- If no interrupts → fetch the next instruction for the current program
- If an interrupt is pending → suspend execution of the current program, and execute the interrupt handler

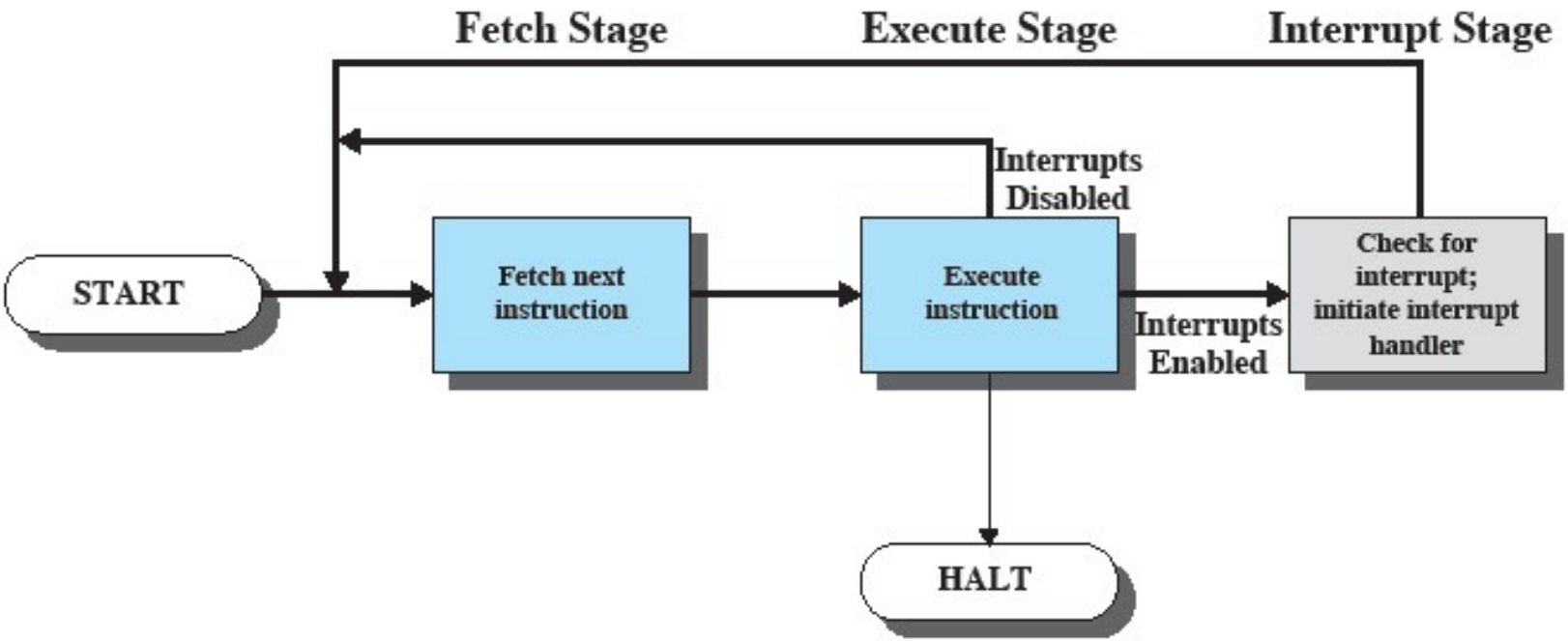
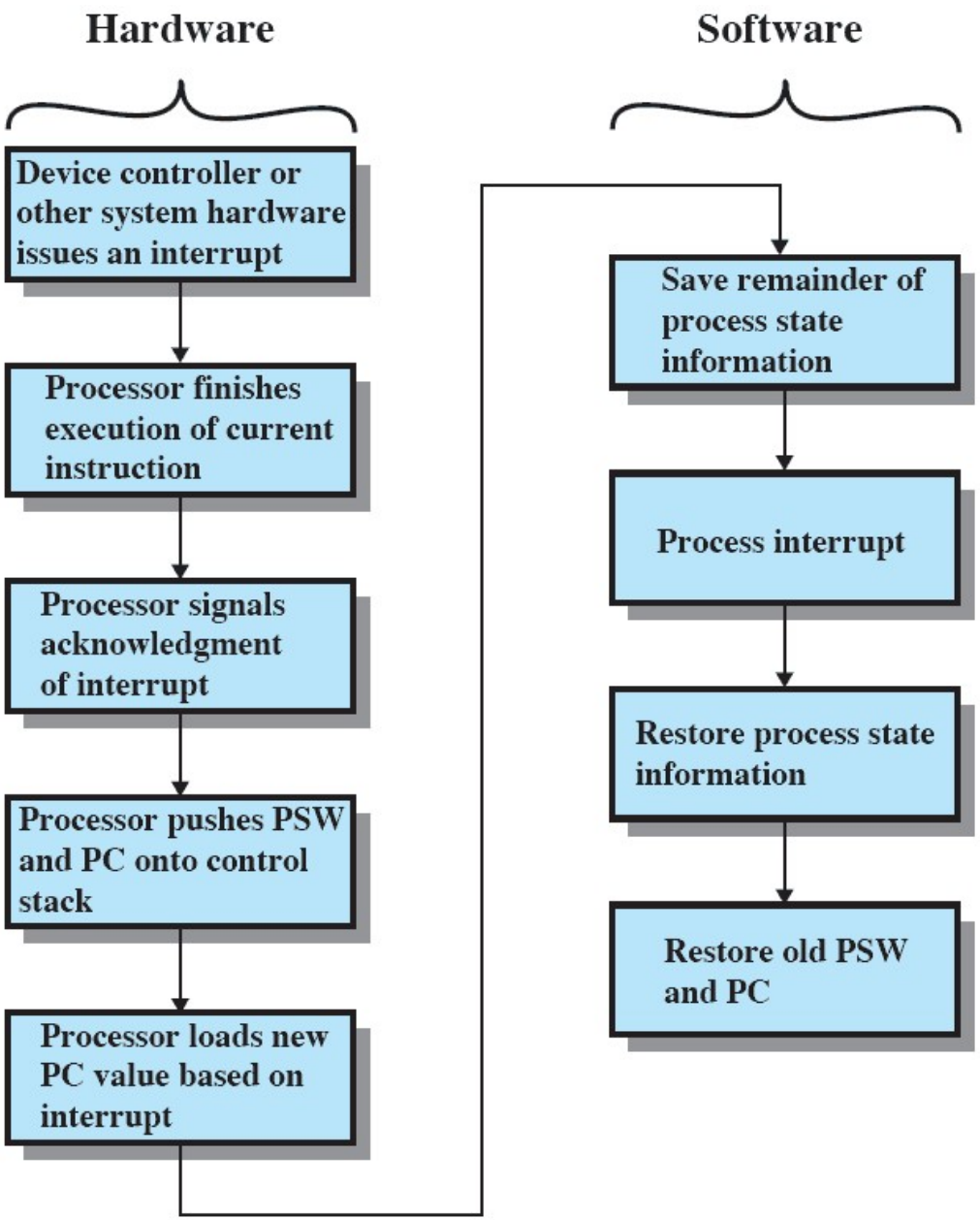
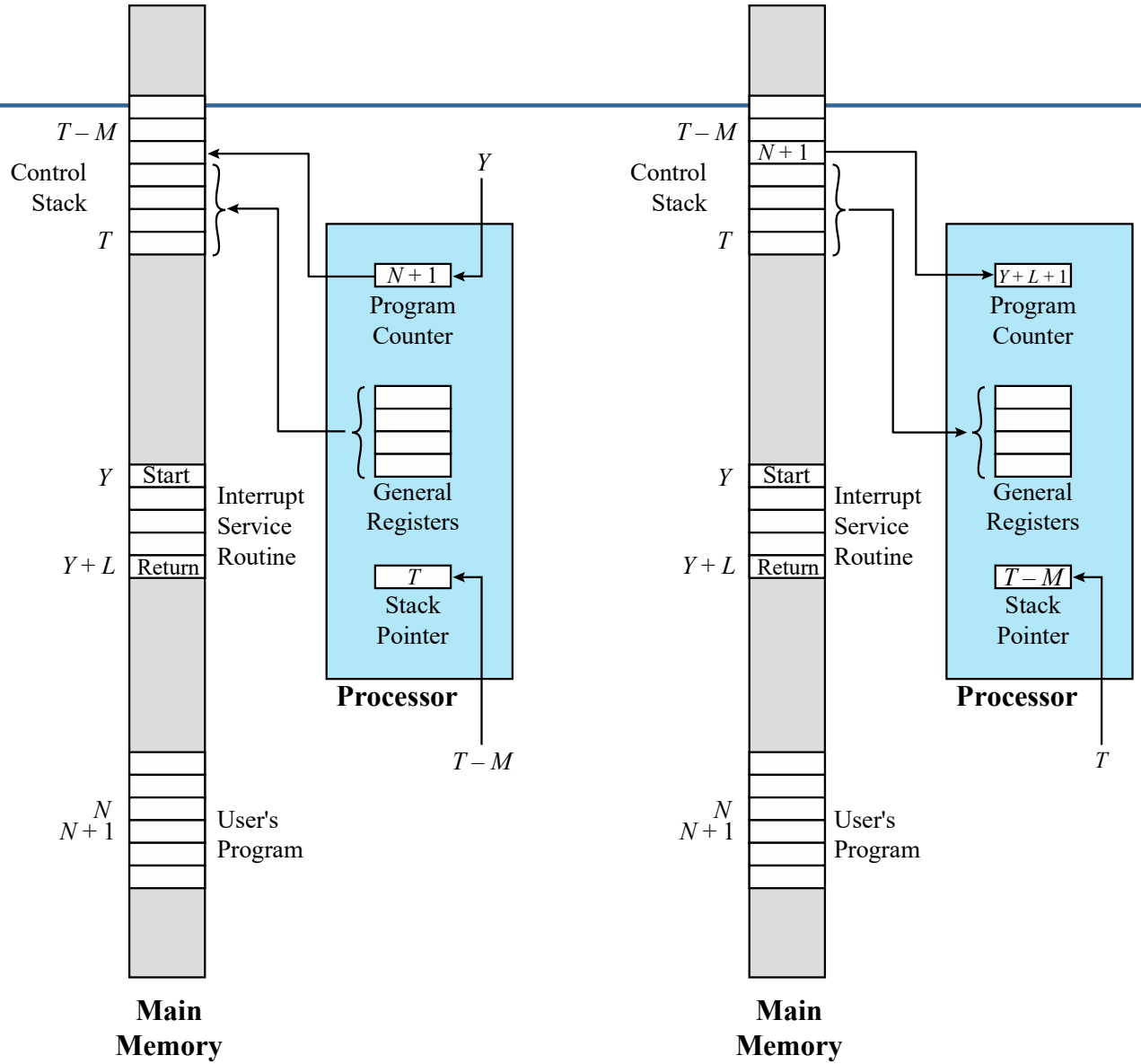


Figure 1.7 Instruction Cycle with Interrupts

Simple Interrupt Processing





(a) Interrupt occurs after instruction at location N

(b) Return from interrupt

Figure 1.11 Changes in Memory and Registers for an Interrupt

Multiple Interrupts

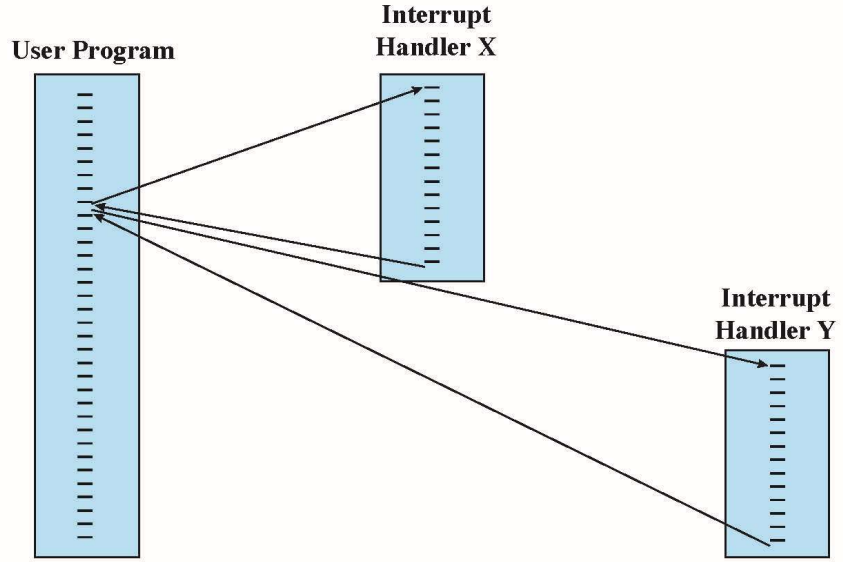
Suppose multiple interrupts received

Solu 1: Disable interrupts during interrupt processing

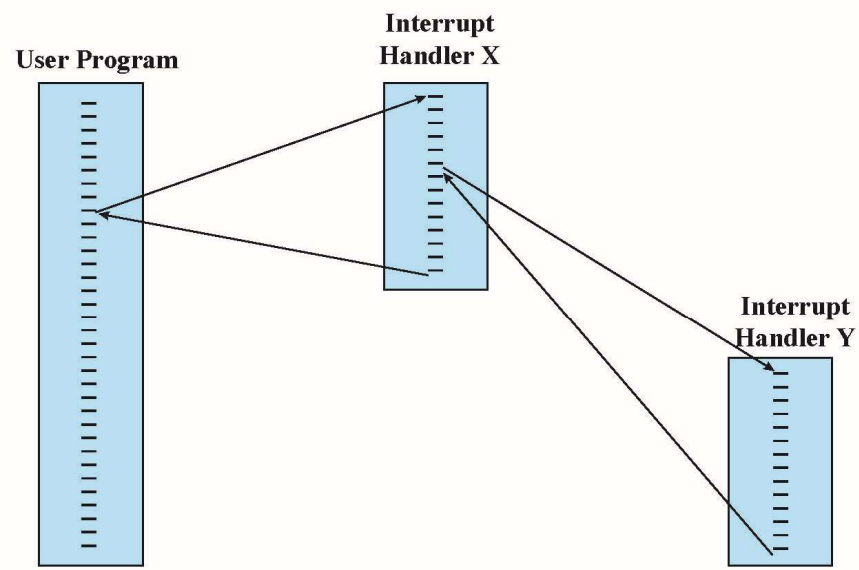
- Interrupts remain pending until the processor enables interrupts
- After interrupt handler routine completes, the processor checks for additional interrupts
- Adv: Simple
- Disadv: Interrupts are handled in strict sequential order; no account for priority

Solu 2: Use a priority scheme

- Lower-priority interrupt handler can be interrupted (nested)



(a) Sequential interrupt processing



(b) Nested interrupt processing

Example of Nested Interrupts

Three I/O devices:
- printer (priority 2)
-disk (priority 4)
- communications line (priority 5)

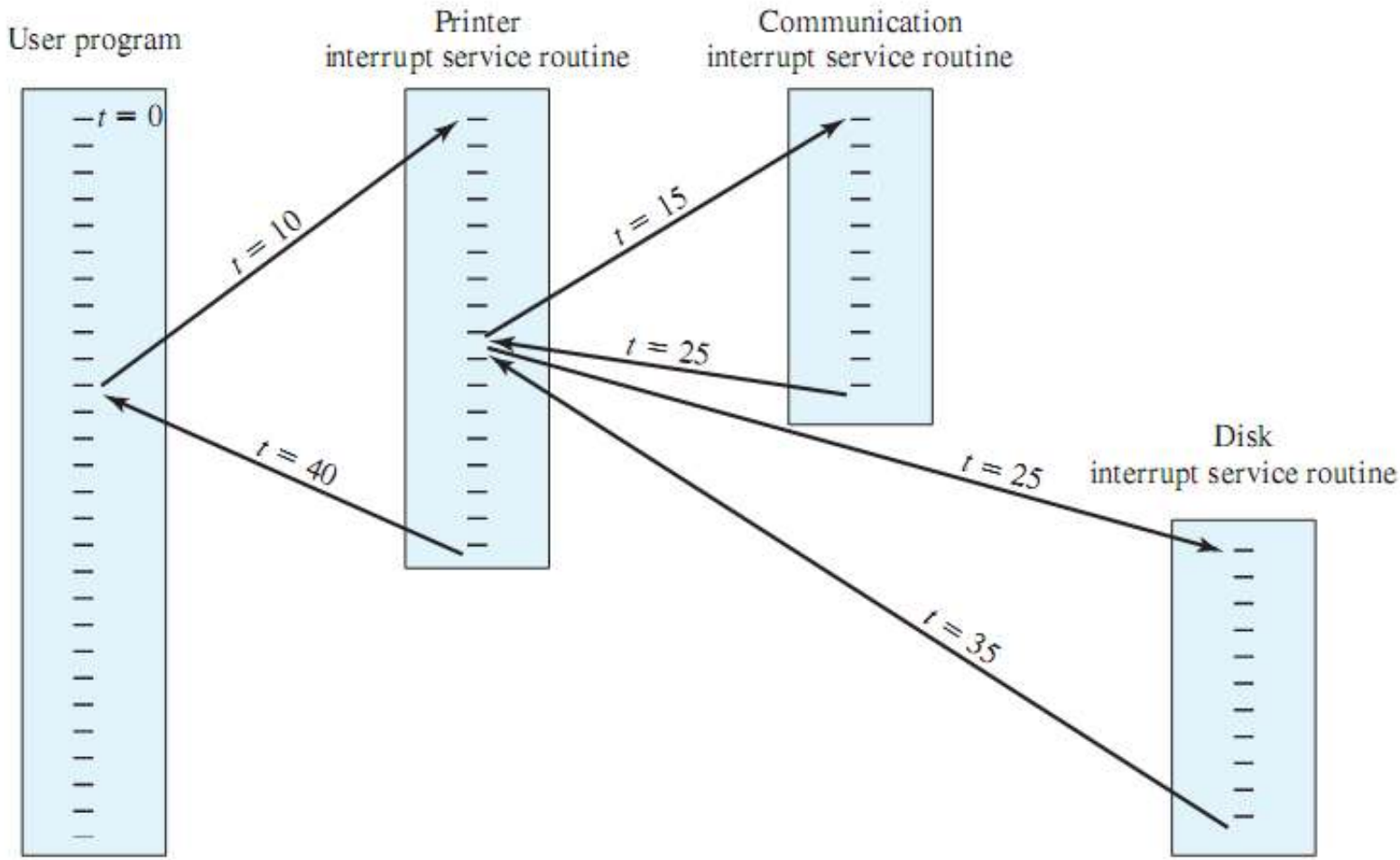


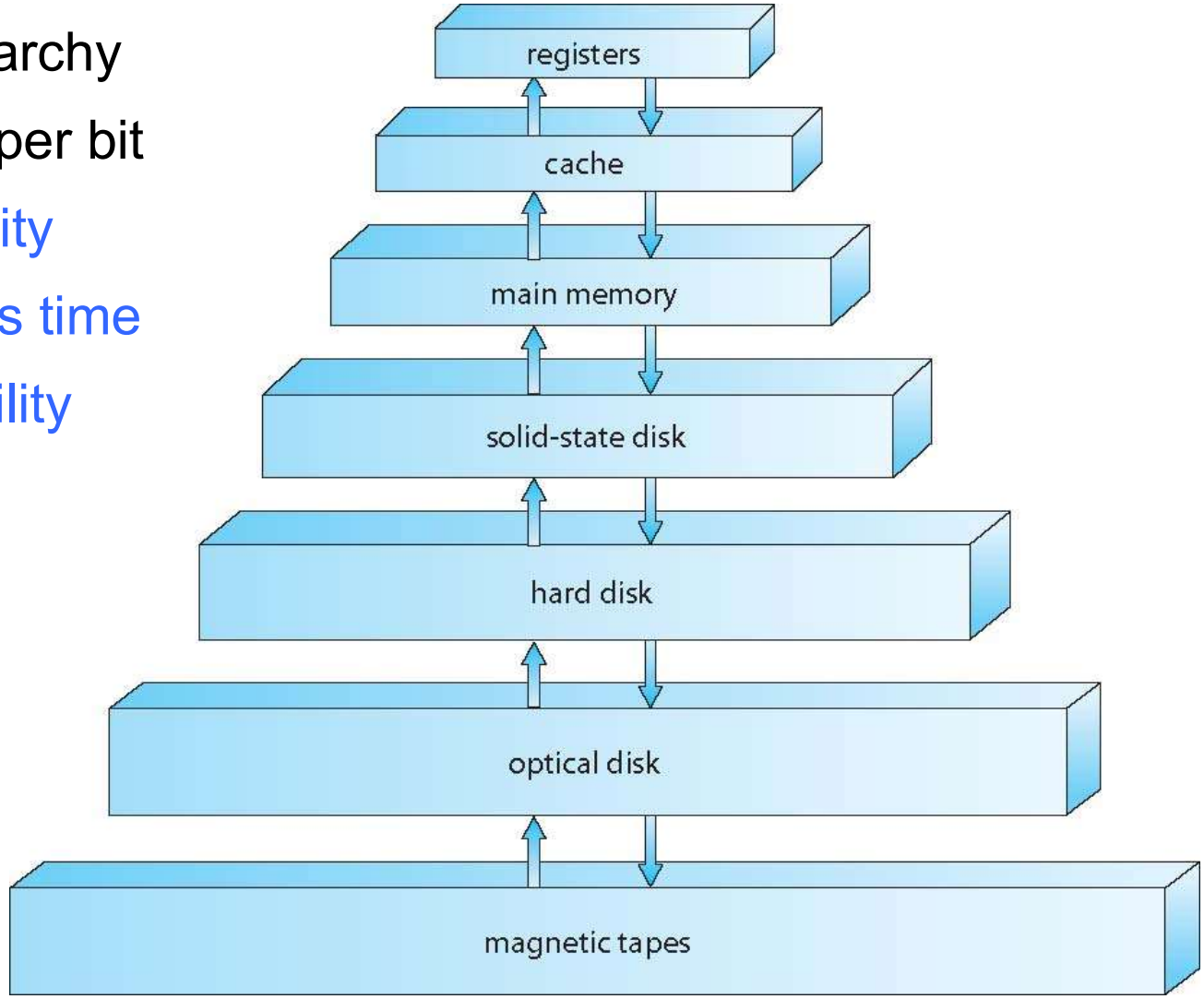
Figure 1.13 Example Time Sequence of Multiple Interrupts

Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, and reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation or to signal a variety of error conditions.
Hardware failure	Generated by a failure, such as power failure or memory parity error

Storage-Device Hierarchy

- Going down the hierarchy
  - Decreasing cost per bit
  - Increasing capacity
  - Increasing access time
  - Decreasing volatility



- Key success trade-off: Decreasing frequency of access to lower level

Performance of Various Levels of Storage

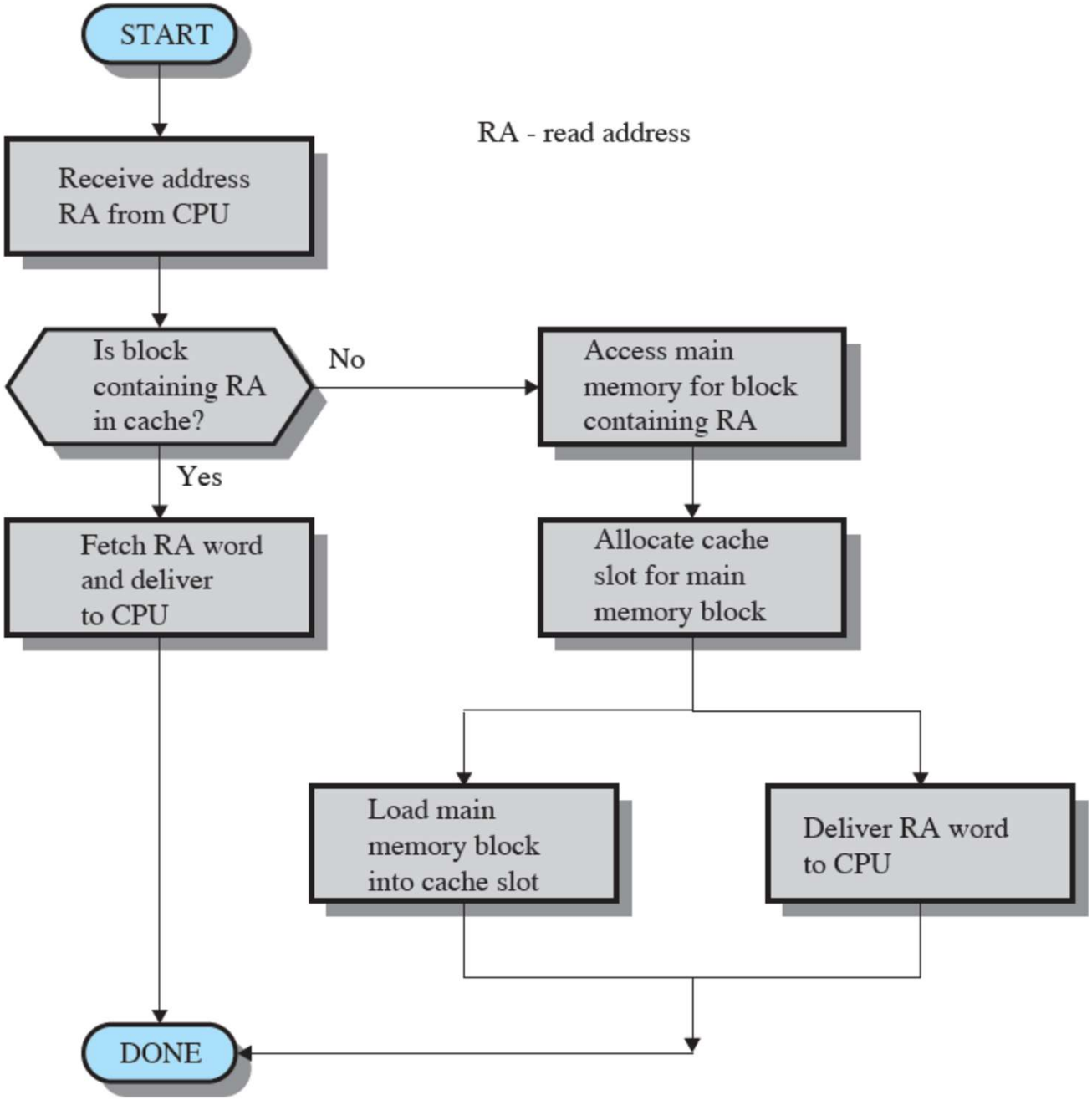
Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Movement between levels of storage hierarchy can be explicit or implicit

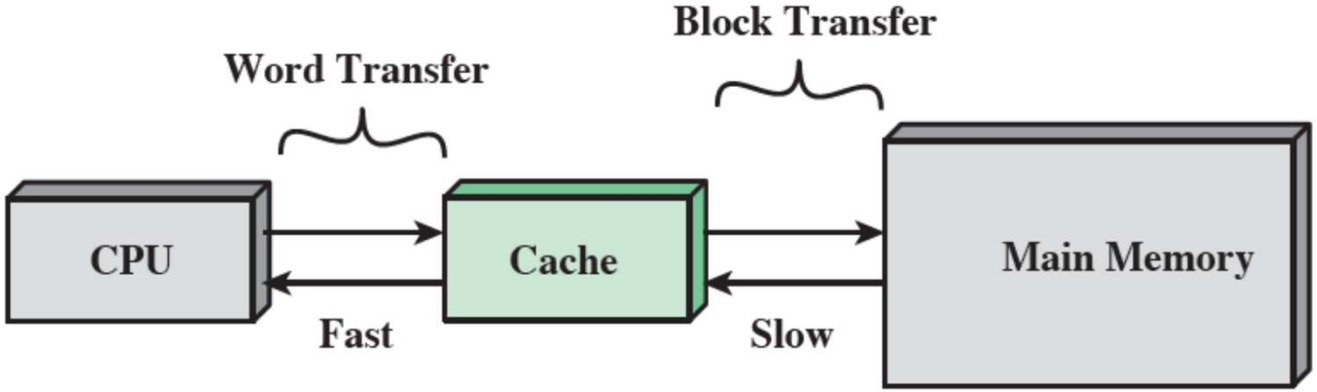
Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is (**hit**), information used directly from the cache (fast)
  - If not (**miss**), data copied to cache and used there
- Cache smaller than storage being cached
  - Cache management important design problem
  - Cache size and replacement policy

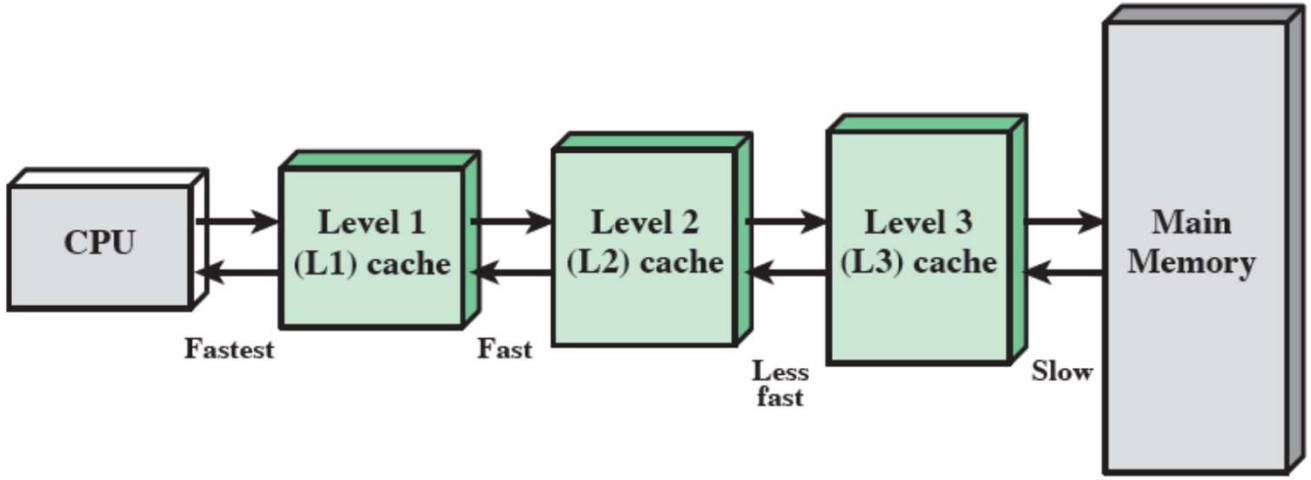
Cache Read Operation



Cache and Main Memory



(a) Single cache



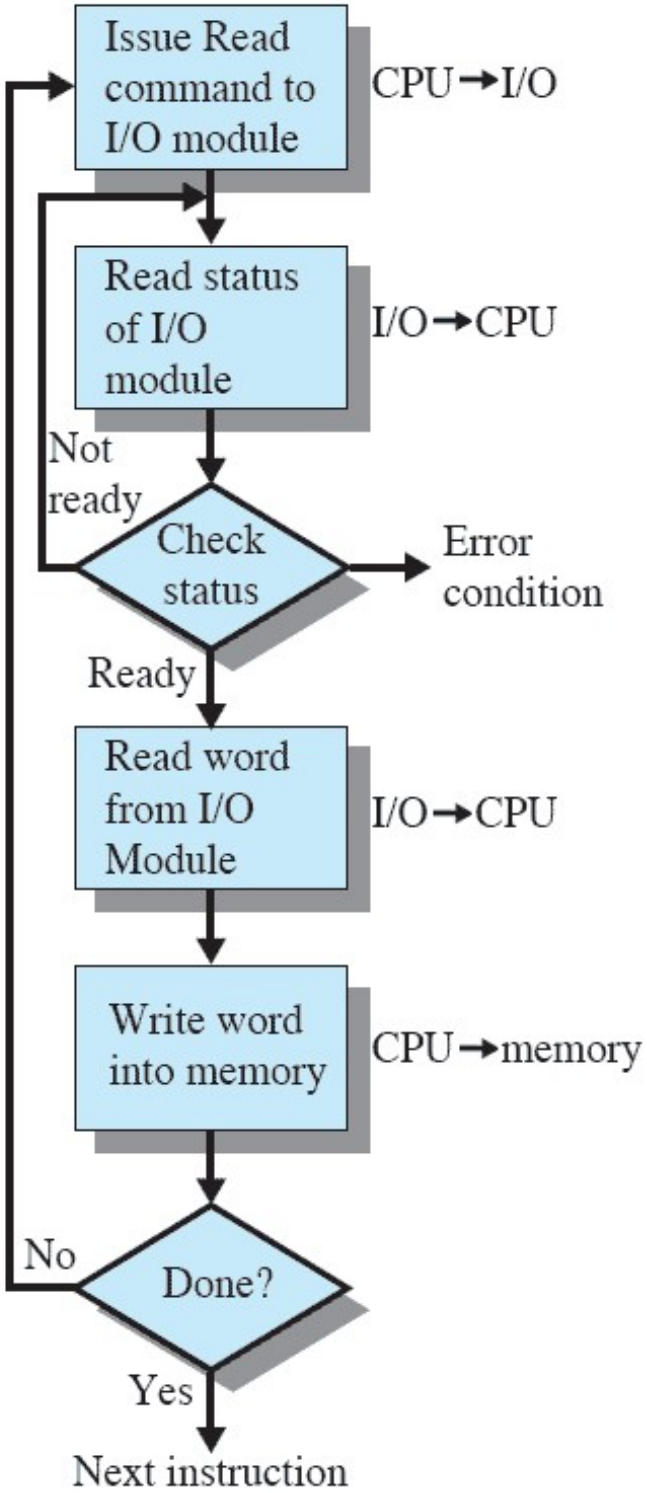
(b) Three-level cache organization

I/O Structure

- When the processor encounters an I/O instruction
  - it executes that instruction by issuing a command to the appropriate I/O module.
- Three techniques are possible for I/O operations:
  - **Programmed I/O**
  - **Interrupt-driven I/O**
  - **Direct memory access (DMA)**

Programmed I/O

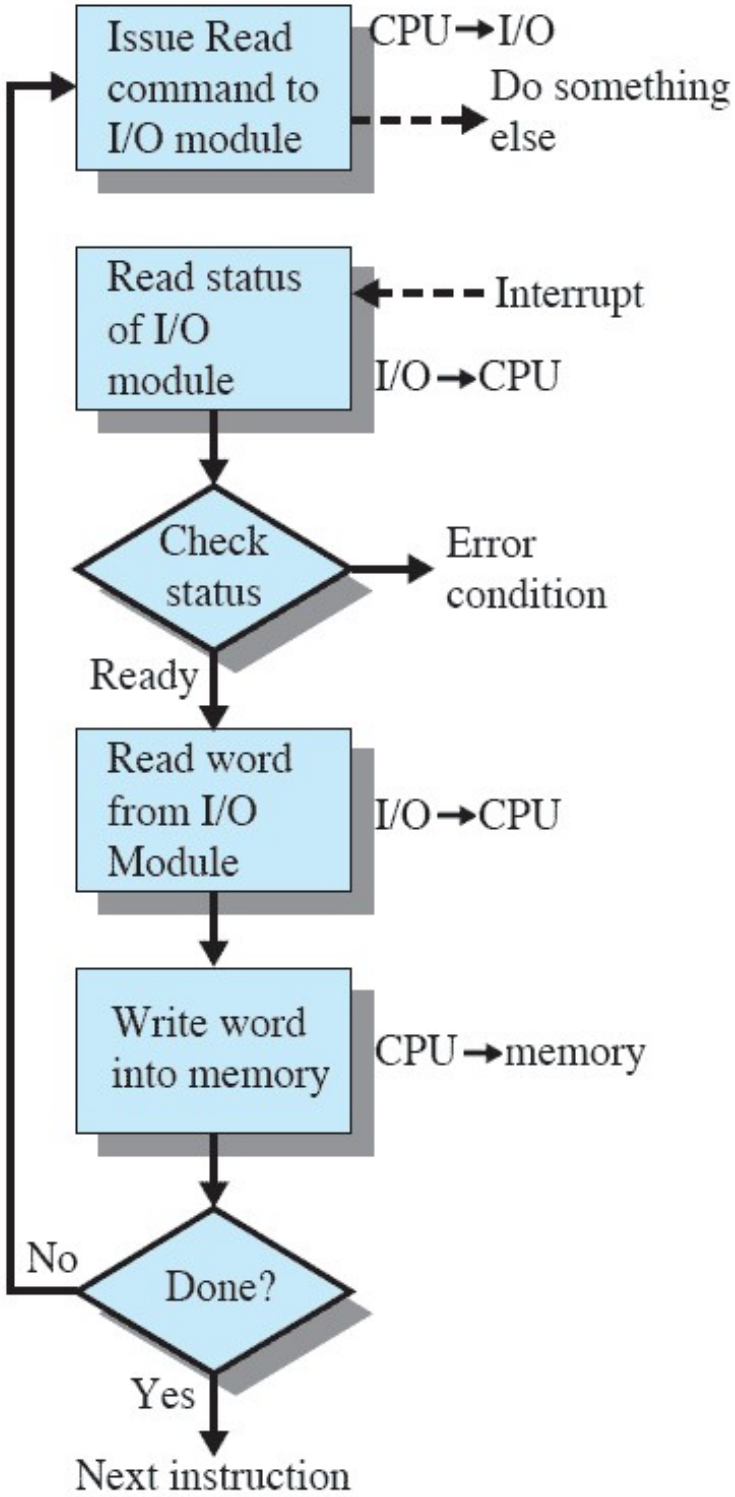
- Upon an I/O request
  - Processor sends request to I/O module
  - I/O module
    - performs the action, not the processor
    - Sets bits in the I/O status register
    - Does not interrupt processor when done
  - Processor must keep checking status
  - Processor is responsible for
    - extracting data from main memory for output
    - storing data in main memory for input
- Disadv: keeps the processor busy needlessly
- Example: read a block of data from external device



(a) Programmed I/O

Interrupt-Driven I/O

- Processor issues an I/O command to a module and then goes on to do some other useful work.
- Processor is interrupted when I/O module ready to exchange data
- Processor is free to do other work
- More efficient than Programmed I/O: No needless waiting
- Disadv: Consumes a lot of processor time because every word read or written passes through the processor



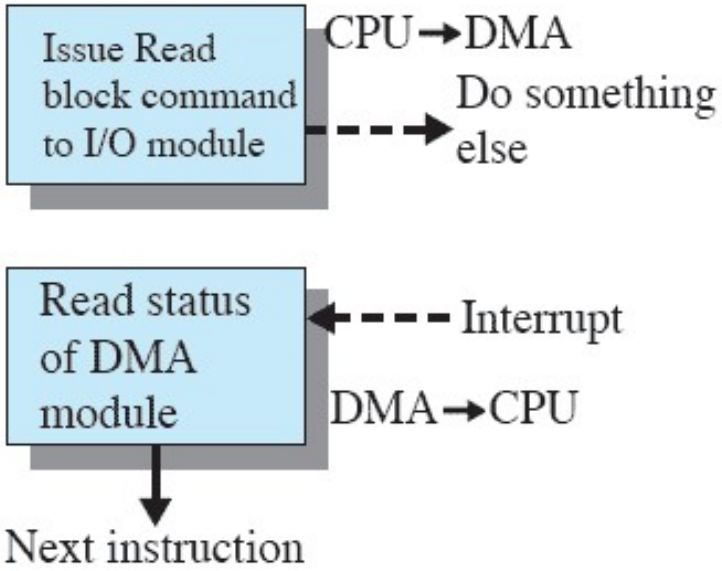
(b) Interrupt-driven I/O

Direct Memory Access

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte
- Processor delegates the I/O operation to DMA
- Efficient for movement of large volumes of data
- Performed by a separate module on the system
- When needing to read/write; processor issues a command to DMA module with:
  - Whether a read or write is requested
  - The address of the I/O device involved
  - The starting location in memory to read/write
  - The number of words to be read/written

Direct Memory Access

- Processor sends I/O request to DMA
- Processor grants I/O module authority to read from or write to memory
- DMA Transfers a block of data directly to or from memory without processor involvement
- When transfer is complete, DMA sends an interrupt signal to the processor
- The processor is only involved at the beginning and end of the transfer.
- Processor is free to do other things
- more efficient, suitable for large volumes of data



(c) Direct memory access

Computer-System Architecture

- **Single-processor Systems**
  - Use a single general-purpose processor
  - Most systems have special-purpose processors as well
    - Disk controller
    - Keyboard controller
    - Graphic controller etc..
- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **multicore systems**
  - They have two or more processors in close communication
  - Processors share the computer bus and sometimes the clock, memory, and peripheral devices.
  - Multiprocessor systems appeared in servers, then migrated to desktop and laptop and recently on mobile devices

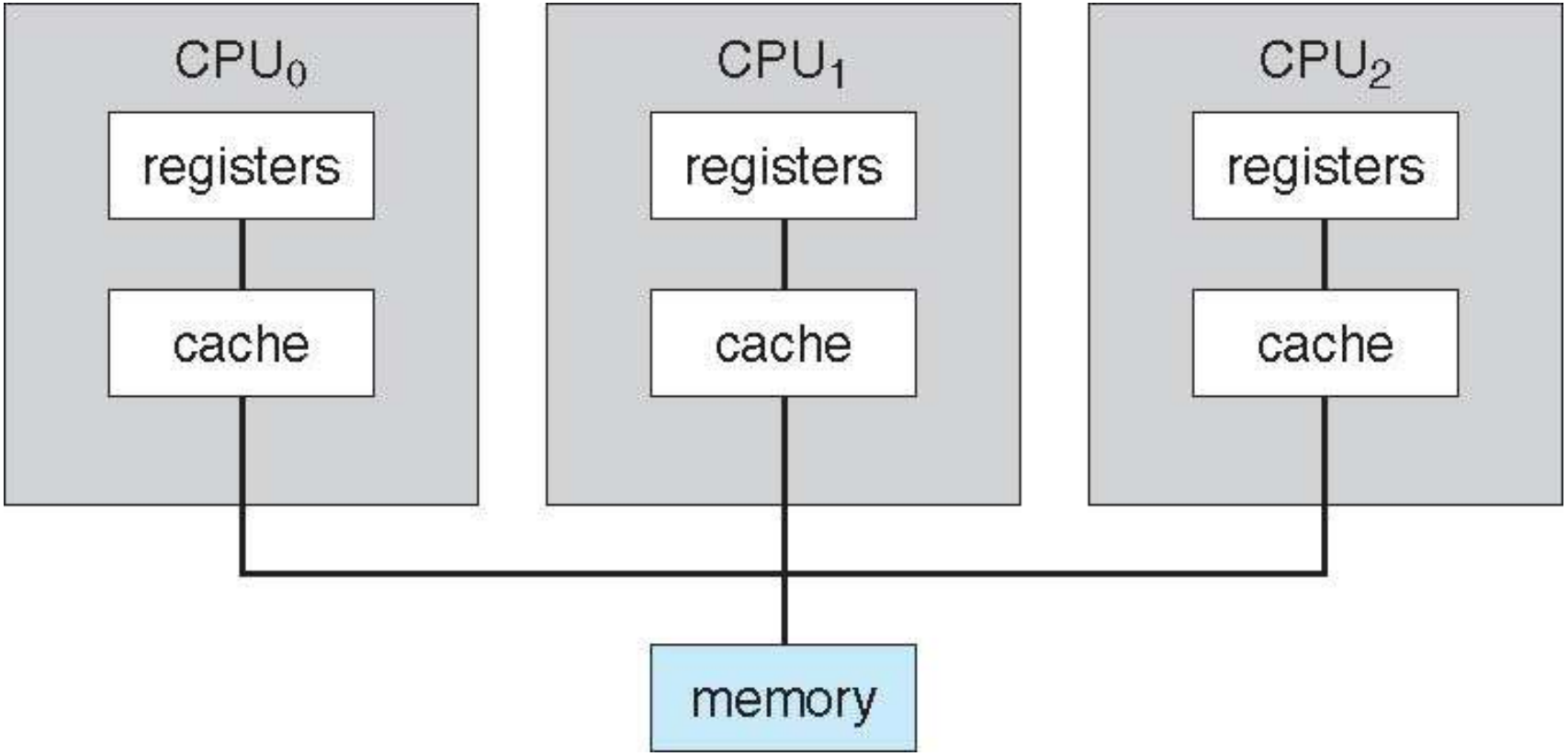
Multiprocessors Systems

- Advantages:
  - **Increased throughput**
    - Increasing number of processors, increase throughput
    - Speed-up ratio with N processors is < N (overhead + contention)
  - **Economy of scale**
    - cost less than multiple single-processor systems, because they can share peripherals, mass storage, and power supplies
  - **Increased reliability**
    - graceful degradation or fault tolerance
- Two types:
  1. **Asymmetric Multiprocessing** – each processor is assigned a specific task. ***Boss–worker*** relationship
     - The boss processor schedules and allocates work to the workers
  2. **Symmetric Multiprocessing** – each processor performs all tasks

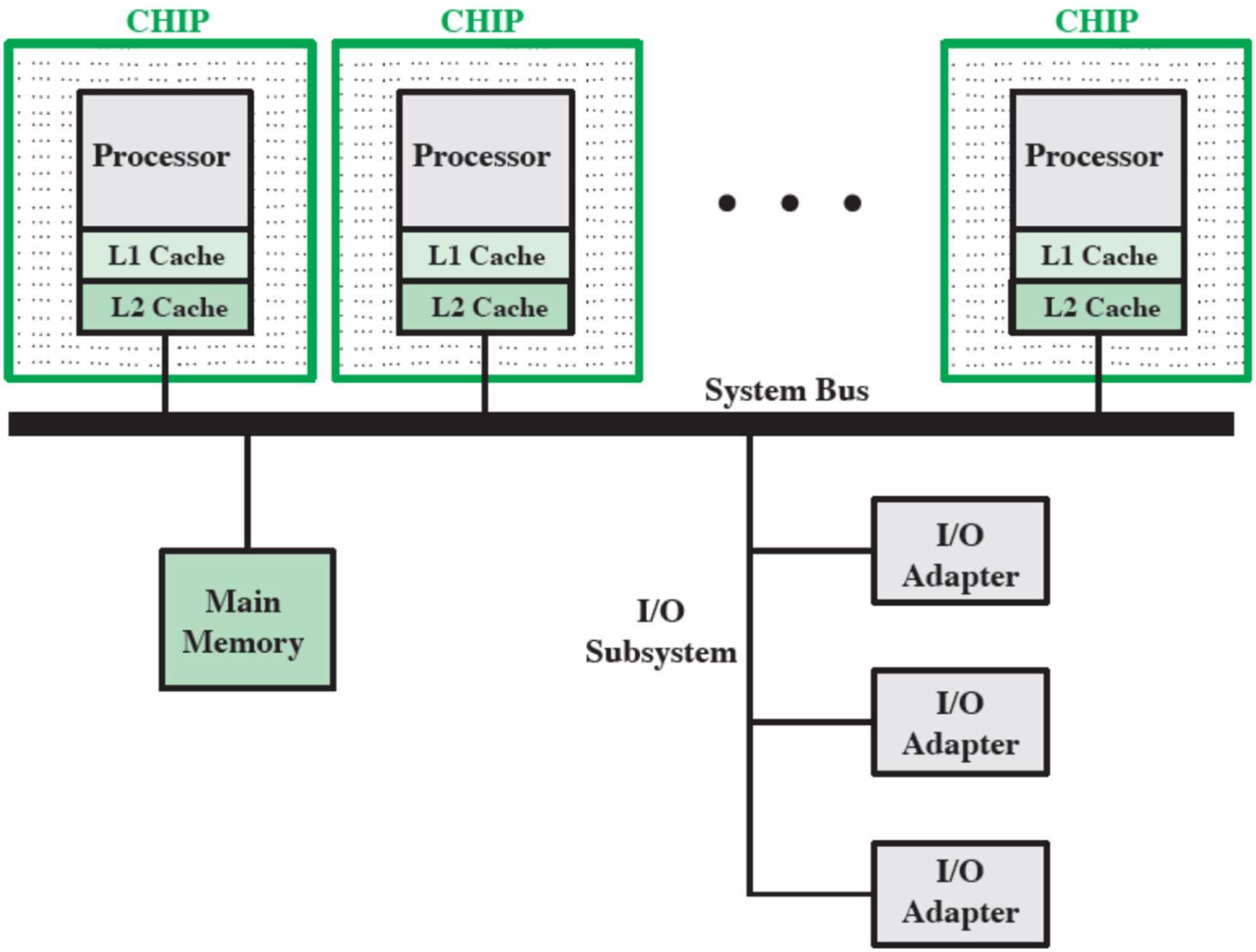
Symmetric Multiprocessor (SMP)

- A stand-alone computer system with the following characteristics:
  - Two or more similar processors of comparable capability
  - Processors share the same main memory and are interconnected by a bus or other internal connection scheme
  - Processors share access to I/O devices
  - All processors can perform the same functions
  - The system is controlled by an integrated operating system that provides interaction between processors and their programs at the job, task, file, and data element levels

Symmetric Multiprocessing Architecture

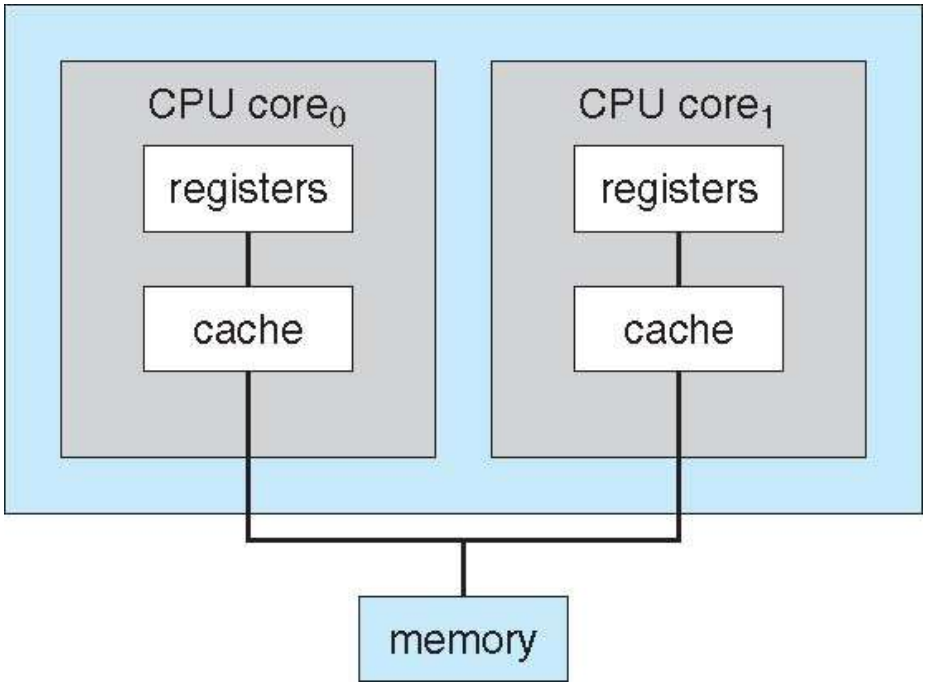


Symmetric Multiprocessor Organisation

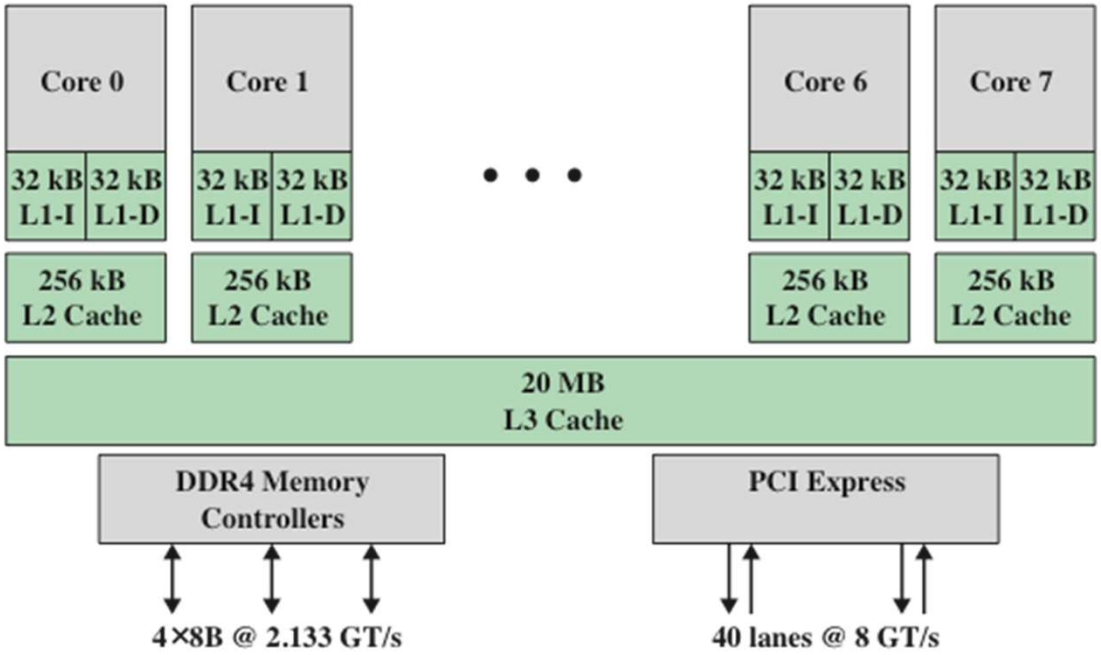


A Dual-Core Design

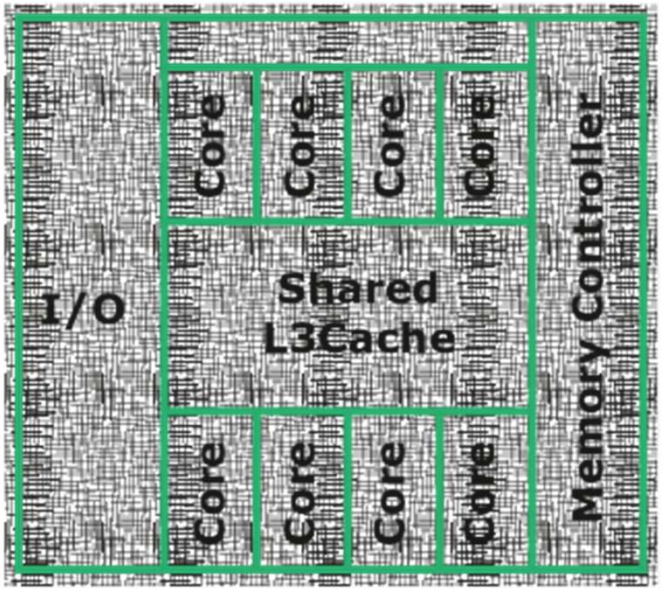
- **Multicore:** CPU include multiple computing cores on a single chip
- More efficient than multiple chips with single core
  - on-chip communication is faster than between-chip communication
  - one chip with multiple cores uses significantly less power than multiple single-core chips.



Intel Core i7-5960X Block Diagram



(a) Block diagram



(b) Physical layout on chip

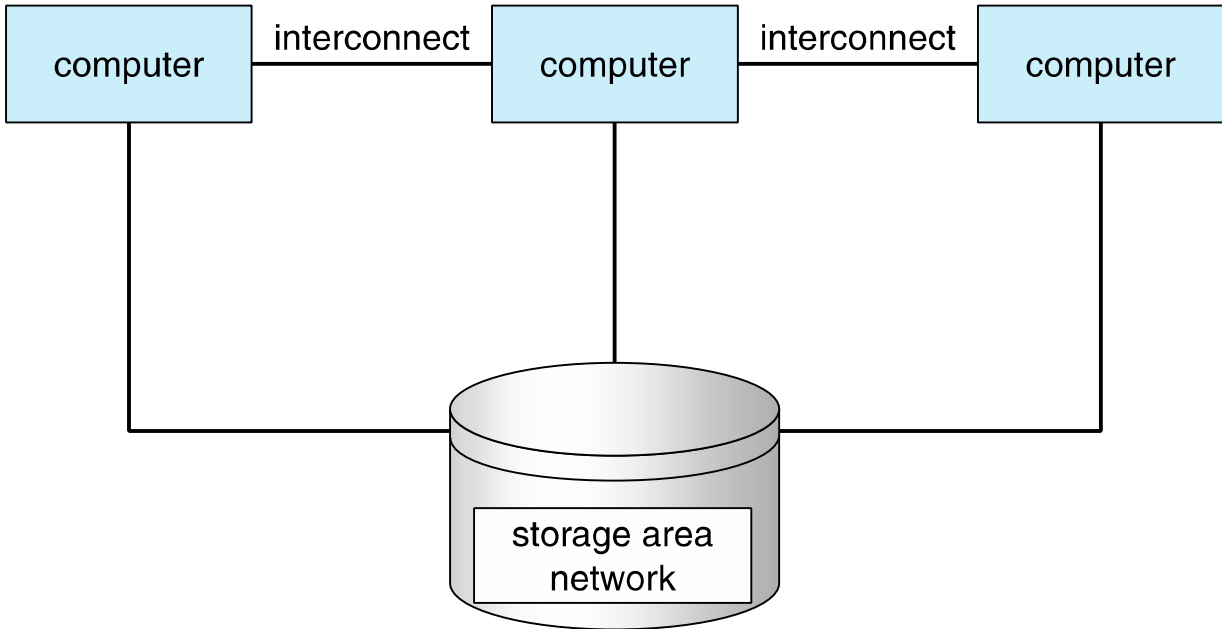
Figure 1.20 Intel Core i7-5960X Block Diagram

SMP Advantages

Performance	More than one process can be running simultaneously, each on a different processor
Availability	Failure of a single process does not halt the system
Incremental Growth	Performance of a system can be enhanced by adding an additional processor
Scaling	Vendors can offer a range of products based on the number of processors configured in the system

Clustered Systems

- Multiprocessor systems with multiple systems/nodes joined together
  - Each node may be a single processor or multicore
  - Closely linked via LAN or InfiniBand
  - Usually sharing storage via a **Storage-Area Network (SAN)**



Clustered Systems

- Provides a **high-availability** service which survives failures
  - A level of redundancy is added: layer of cluster software runs on the cluster nodes
  - Each node can monitor one or more of the others (over the LAN).
  - If the monitored machine fails, the monitoring machine can take ownership of its storage and restart the applications that were running on the failed machine.
- **Asymmetric clustering** has one machine in hot-standby mode; others running the applications
  - The hot-standby host machine does nothing but monitor the active servers
  - If that server fails, the hot-standby host becomes the active server.
- **Symmetric clustering** has multiple nodes running applications, monitoring each other
  - more efficient, as it uses all of the available hardware. However it does require that more than one application be available to run.
- Some clusters are for **high-performance computing (HPC)**
  - can run an application concurrently on all computers in the cluster
  - Applications must be written to use parallelization